

Figure 3.3 Output from running the API server

Here is an overview of how to perform these tasks:

- *Stop the server.* If the server is running, press Ctrl-C in the terminal window to stop it.
- *Start the server.* If the server isn't running, type `node app.js` in the terminal window to start it.
- *Restart the server.* Stop the server, then start the server (that is, press Ctrl-C followed by typing `node app.js`).

You can verify that the server is working by visiting http://127.0.0.1:9999/api/posts in a browser. You should see a JSON response similar to figure 3.4. If you don't see this response or you receive an error, review the preceding steps to make sure everything is in order.

Now that you have a working API, let's build the page that uses this API.

3.1.2 Setting up the sample client

With the API in place, you're ready to add the client.html page that consumes the API, as highlighted in figure 3.5.

Copy the contents of listing 3.2 into a new file named client.html, and save it in the same directory as app.js. Client.html is the web page that reads the data from the API and displays it in the browser.

Chapter 2 covered the basics of how the client makes CORS requests, but here is a recap of the code in listing 3.2. The main functionality of client.html takes place in the `getBlogPost` function. The function starts by creating a new `XMLHttpRequest` object. `XMLHttpRequest` is the standard mechanism for making HTTP requests in

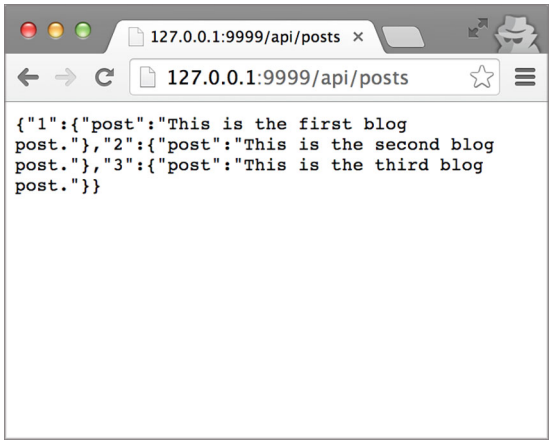


Figure 3.4 The response from a working server